# Python Chapter 6 Lists Activities

## Contents


Python Chapter 6 Lists Activities ..... 1
DRY ..... 1
Online Tutorials ..... 2
Traversing a List with a Loop ..... 2
Slice ..... 3
List Functions ..... 4
    Examples ..... 5
    in ..... 5
List Methods ..... 5
    append() ..... 6
    insert() ..... 6
    sort() ..... 6
    reverse() ..... 6
    index() ..... 6
    remove() ..... 7
    del ..... 7
    pop() ..... 7
    extend() ..... 7
Two-dimensional Lists ..... 8
Tutorial 1: Two-Dimensional Nested List ..... 9
Try Except ..... 11
Tutorial 4: Try Except Fahrenheit Converter ..... 12
  Assignment Submission ..... 13


Time required: 90 minutes

# DRY

Don’t Repeat Yourself

## Online Tutorials

Go through the following tutorials.

- LearnPython.org Lists
- Python Lists
    - Access List Items
    - Change List Items
    - Add List Items
    - Remove List Items
    - Loop Lists
    - List Comprehensions
    - Sort Lists
    - Copy Lists
    - Join Lists
    - List Methods

## Traversing a List with a Loop

The most common way to iterate or traverse the elements of a list is with a for loop. The syntax is the same as for strings.

```
for cheese in cheeses:
    print(cheese)
cheddar
colby
swiss
```

If you want to write or update the elements, you need the indices. A common way to do that is to combine the functions **range** and **len**:

```
# range_len.py
numbers = [1, 3, 5]
for i in range(len(numbers)):
    # Update element in list
    numbers[i] = numbers[i] * 2
    print(f"{i + 1} {numbers[i]}")
```

Example run:

```
1 2
2 6
3 10
```

This loop traverses the list and updates each element. **len** returns the number of elements in the list. **range** returns a list of indices from 0 to n − 1, where **n** is the length of the list. Each time through the loop, **i** gets the index of the next element. The assignment statement in the body uses **i** to read the old value of the element and to assign the new value.

**Nested List:** Although a list can contain another list, the nested list still counts as a single element. The length of this list is four:

```
["spam", 1, ["Brie", "Roquefort", "Pol le Veq"], [1, 2, 3]]
```

## Slice

The Python slice operator, represented by the colon **:**, allows you to extract a portion of a sequence like a string, list, or tuple by specifying a start, stop, and step size within square brackets [ ].

The general syntax for slicing is: **sequence[start:stop:step]**

- **start (optional)**: The index where the slice begins (inclusive). If omitted, it starts from the beginning of the sequence (index 0).
- **stop (optional)**: The index where the slice ends (exclusive). If omitted, it goes until the end of the sequence.
- **step (optional)**: The increment between indices while slicing. If omitted, it defaults to 1.

```
t = ["a", "b", "c", "d", "e", "f"]
print(t[1:3])
# Output: ['b', 'c']
print(t[:4])
# Output: ['a', 'b', 'c', 'd']
print(t[3:])
# Output: ['d', 'e', 'f']
```

- `t[1:3]` slices the index 1 and 2 items.
- `t[:4]` slices everything from the beginning of the list to index 3.
- `t[3:]` slices everything after and including index 3.

If you omit the first index, the slice starts at the beginning. If you omit the second, the slice goes to the end. If you omit both, the slice is a copy of the whole list.

```
print(t[:])
# Output: ['a', 'b', 'c', 'd', 'e', 'f']
```

As lists are mutable, it is often useful to make a copy before performing operations that fold, spindle, or mutilate lists.

**slice** selects all the elements up to, but not including the second index.

## List Functions

There are many built-in functions that operate on lists without writing your own loops.

| Function | Description |
|---|---|
| **`len()`** | Returns the number of items in the list |
| **`sum()`** | Returns the sum of the items in the list |
| **`min()`** | Returns the minimum of the items in the list |
| **`max()`** | Returns the maximum of the items in the list |
| **`sorted()`** | Returns a sorted list |

### Examples

```
nums = [3, 41, 12, 9, 74, 15]
print(len(nums))
# Output: 6
print(max(nums))
# Output: 74
print(min(nums))
# Output: 3
print(sum(nums))
# Output: 154
print(sum(nums)/len(nums))
# Output: 25
print(sorted(nums))
# Output: [3, 9, 12, 15, 41, 74]
```

### in

The in operator tells you if a list contains something.

```
if 2 in L:
   print("Your list contains the number 2. ")
if 0 not in L:
   print("Your list has no zeroes.")
```

## List Methods

Python provides methods that operate on lists. Methods use the **.** (dot) operator.

| Method | Description |
|---|---|
| **`append(x)`** | add **x** to the end of the list |
| **`sort(x)`** | sort the list |
| **`count(x)`** | return the number of times **x** occurs in the list |
| **`index(x)`** | return the location of the first occurrence of **x** |
| **`reverse()`** | reverse the list |
| **`clear()`** | remove all the elements from the list |
| **`remove(x)`** | remove the first occurrence of **x** from the list |

| | |
|---|---|
| **`pop(p)`** | remove the item at index **p** and return its value |
| **`insert(p, x)`** | insert **x** at index **p** of the list |

### append()

Append adds a new element to the end of a list:

```
t = ["a", "b", "c"]
t.append('d')
print(t)
# Output: ['a', 'b', 'c', 'd']
```

### insert()

Insert places a new element at the designate index and moves the others aside.

```
t = ["a", "b", "c"]
t.insert(1, "d")
print(t)
# Output: ['a', 'd', 'b', 'c']
```

### sort()

Arranges the elements of the list from low to high:

```
t = ["d", "c", "e", "b", "a"]
t.sort()
print(t)
# Output: ['a', 'b', 'c', 'd', 'e']
```

### reverse()

The reverse() reverses the original order of the list. It doesn't sort backward alphabetically, it reverses the current order of the list,

```
t = ["d", "c", "e", "b", "a"]
t.reverse()
print(t)
t.sort()
print(t)
# Output: ['d', 'c', 'e', 'b', 'a']
# Output: ['a', 'b', 'c', 'd', 'e']
```

### index()

Returns the index of the first occurrence on the list of the element that is given to index() as argument. A runtime error is generated if the element is not found on the list.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
print(fruitlist.index("banana"))
# Output: 1
```

### count()

Returns an integer that indicates how often the element that is passed to it as an argument occurs in the list.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
print(fruitlist.count("banana"))
# Output: 2
```

### remove()

Removes an item with a certain value.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
fruitlist.remove("banana")
print(fruitlist)
# Output: ['apple', 'cherry', 'banana', 'durian']
```

### del

**del** is a function, not a method. **del** removes an item at an index.

```
t = ["a", "b", "c"]
del t[1]
print(t)
# Output: ['a', 'c']
```

### pop()

**pop** removes an item from a list at a designated index and returns a value. You can assign that value to a variable. If you don’t provide an index, it deletes and returns the last element.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
popfruit = fruitlist.pop(1)
print(fruitlist)
print(popfruit)
# Output: ['apple', 'cherry', 'banana', 'durian']
# Output:Banana
```

### extend()

Takes a list as an argument and appends all the elements. This example leaves t2 unmodified.

```
t1 = ["a", "b", "c"]
t2 = ["d", "e"]
t1.extend(t2)
print(t1)
# Output: ['a', 'b', 'c', 'd', 'e']
```

**join()**

.join() is not a list method, it is a string method. It does work very well with lists. .join() iterates through the list joining each string with the separator.

```
fruitlist = ["apple", "banana", "cherry", "banana", "durian"]
separator = " | "
output = separator.join(fruitlist)
print(output)
# Output:apple | banana | cherry | banana | durian
```

## Two-dimensional Lists

There are several common things that can be represented by two-dimensional lists, like a Tictac-toe board or the pixels on a computer screen. In Python, one way to create a two-dimensional list is to create a list whose items are themselves lists. Here is an example:

```
board = [[1,2,3],
         [4,5,6],
         [7,8,9]]
```

**Indexing:** We use two indices to access individual items. To get the entry in row r, column c, use the following:

```
board[r][c]
```

**Printing a two-dimensional list:** To print a two-dimensional list, you can use nested for loops.

The following example prints a 3 x 3 list:

```
for row in range(3):
    for column in range(3):
        print(board[row][column], end=" ")
    print()
```

Example run:

```
1 2 3
4 5 6
7 8 9
```

Another option is to use the **pprint** function of the **pprint** module. This function is used to "pretty-print" its argument. Here is an example to print a list L:

```
from pprint import pprint
pprint(board)
```

The **`pprint`** function can be used to nicely print ordinary lists and other objects in Python.

**Working with two-dimensional lists:** Nested for loops, like the ones used in printing a two-dimensional list, can also be used to process the items in a two-dimensional list. Here is an example that counts how many entries in a 10 x 5 list are even.

```
count = 0
for row in range(10):
    for column in range(5):
        if L[row][column]%2==0:
            count = count + 1
```

**Creating large two-dimensional lists:** To create a larger list, you can use a list comprehension like below:

```
L = [[0]*50 for i in range(100)]
```

This creates a list of zeroes with 100 rows and 50 columns.

**Picking out rows and columns:** To get the rth row of L, use the following:

```
L[r]
```

To get the cth column of L, use a list comprehension:

```
[L[i][c] for i in range(len(L))]
```

## Tutorial 1: Two-Dimensional Nested List

Nested lists are lists that contain other lists as their elements. They allow you to create multi-dimensional data structures.

This tutorial provides explanations and examples for creating, accessing, modifying, and iterating through a two-dimensional nested list in Python.

```python
"""
    Name: nested_list.py
    Author:
    Created:
    Purpose:
"""

# Create a nested list
# Create a variable 'matrix' and assign it a list containing other list
matrix = [
    [1, 2, 3],     # First nested list with elements 1, 2, and 3.
    [4, 5, 6],     # Second nested list with elements 4, 5, and 6.
    [7, 8, 9]      # Third nested list with elements 7, 8, and 9.
]

# Access elements in the nested list
# Print the element at the first row, first column (1).
print(matrix[0][0])
# Print the element at the second row, third column (6).
print(matrix[1][2])

# Modify elements in the nested list
# Change the element at the third row, second column to 10.
matrix[2][1] = 10
# Print the modified element (10).
print(matrix[2][1])

# Iterate through the nested list
# Start a loop that iterates through the outer list (rows).
for row in matrix:
    # Start a nested loop that iterates through
    # the inner list (elements in each row).
    for item in row:
        # Print each element followed by a space, without a new line.
        print(item, end=' ')
    # Print a new line after each row.
    print()
```

Example run:

```
# Output:
# Accessing Elements:
# 1
# 6

# Modifying Elements:
# 10

# Iterating Through the List:
# 1 2 3
# 4 5 6
# 7 10 9
```

```
1
6
10
1 2 3
4 5 6
7 10 9
```

## Try Except

Things go wrong, errors happen, let's learn to handle those exceptional events with try except.

An exception is a signal that a condition has occurred that can't be easily handled using the normal flow-of-control of a Python program. Exceptions are often defined as being "errors" but this is not always the case. All errors in Python are dealt with using exceptions, but not all exceptions are errors.

Go through this short tutorial for a better understanding of exceptions.

- Python Try Except

Basic syntax of exception handling

```
try:
    # Code that might raise an exception
except ExceptionType as e:
    # Code to handle the exception
```

Example of

Here is an example of a program without exception handling.

```
imput = input('Enter Fahrenheit Temperature: ')
fahrenheit = float(input)
celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
print(celsius)
```

If the user types in a float, the program works just fine. If the user types in anything else, a **ValueError** exception occurs.

Example run:

```
Enter Fahrenheit Temperature: d
Traceback (most recent call last):
  File "Z:\_WNCC\Computer Science\Assignments Python\0
aries\Tutorials\temperature_converter.py", line 2, in
    fahrenheit = float(input)
ValueError: could not convert string to float: 'd'
```

Let's rewrite this program to catch and handle exceptions.

## Tutorial 4: Try Except Fahrenheit Converter

Create and test the following program with correct and incorrect input.

```python
"""
    Name: temperature_converter.py
    Author:
    Created:
    Purpose: Handle ValueError and general exceptions
"""

try:
    # If the user doesn't type a number,
    # an exception is raised
    fahrenheit = float(input("Enter Fahrenheit Temperature: "))

    # These lines execute if an exception isn't raised
    celsius = ((fahrenheit - 32.0) * 5.0) / 9.0
    print(celsius)

except ValueError as v:
    # This is a specific exception
    # You don't have to have a specific exception handler
    # Handle ValueError, print out exception
    print(f"Please enter a whole number. \n{v}")

except Exception as e:
    # Handle any other exception, print out exception message
    # You can just use this general exception handling
    print(f"Something went wrong. \n{e}")
```

Exceptions are now handled.

Example run:

```
Enter Fahrenheit Temperature: b
Please enter a whole number.
could not convert string to float: 'b'
```

## Assignment Submission

1. Attach the program files.
2. Attach screenshots showing the successful operation of the programs.
3. Submit in Blackboard.